ABSTRACT

The proposed system is a Smart Network Traffic Monitoring and Intrusion Detection System for healthcare networks designed to enhance the security of hospital information systems by continuously monitoring network traffic and identifying potential cyber threats in real time. Modern healthcare environments rely on interconnected systems such as Electronic Health Record (EHR) systems, hospital databases, laboratory systems, patient portals, pharmacy systems, and network-connected medical devices. The increasing use of these technologies creates a need for intelligent security mechanisms capable of protecting sensitive healthcare information and maintaining the availability of essential medical services. The proposed system utilizes machine-learning techniques to analyze healthcare network traffic patterns, detect anomalies, and classify malicious activities such as denial-of-service (DoS) attacks, unauthorized access, port scanning, and other network intrusions. By intelligently distinguishing between normal and suspicious network traffic, the system enables the early detection of potential security threats while reducing false-positive alerts. The application captures healthcare network traffic, extracts relevant network features, and processes the collected information using a machine-learning classification model to identify abnormal behavior. A web-based monitoring dashboard provides real-time information regarding network activity, connected healthcare systems, detected threats, traffic statistics, and security alerts. When suspicious activity is identified, the system generates an automated alert containing information such as the attack type, affected healthcare system, threat severity, source address, and detection time. This enables hospital network administrators to respond promptly and take appropriate security measures. The proposed system can be deployed in hospitals, healthcare organizations, diagnostic laboratories, telemedicine platforms, and cloud-based healthcare environments. The primary objective of the project is to provide an intelligent, accurate, and efficient intrusion detection solution that addresses the limitations of traditional signature-based security systems. By integrating artificial intelligence with healthcare network traffic analysis, the proposed system enhances cyber-threat detection, reduces incident-response time, protects sensitive patient information, and contributes to the development of a secure and resilient digital healthcare environment.

CONTENTS

Chapter-1

INTRODUCTION

The rapid advancement of digital technologies has significantly transformed the healthcare sector by improving the accessibility, efficiency, and quality of healthcare services. Modern healthcare organizations increasingly depend on computer networks and interconnected digital systems for managing Electronic Health Records (EHRs), patient information, medical reports, laboratory data, pharmacy services, hospital administration, telemedicine platforms, cloud-based healthcare applications, and network-connected medical devices. These technologies enable healthcare professionals to access and exchange critical information efficiently, support faster clinical decision-making, and provide improved patient care. However, the growing dependence on interconnected healthcare networks has also increased the risk of cyberattacks and security breaches. Healthcare networks store and transmit large amounts of sensitive and confidential information, including patient records, medical histories, diagnostic reports, treatment information, personal details, and financial data. The sensitive nature of this information makes healthcare organizations attractive targets for cybercriminals. Cyber threats such as malware attacks, ransomware, denial-of-service (DoS) attacks, unauthorized access, phishing, data breaches, port scanning, and network intrusions can compromise patient information, disrupt hospital operations, affect the availability of healthcare services, and create significant financial and reputational losses. Attacks on critical hospital systems and connected medical devices may also affect the continuity and reliability of essential healthcare services.

Traditional network security mechanisms, such as firewalls, antivirus software, and signature-based Intrusion Detection Systems (IDS), provide protection against known cyber threats by comparing network activities with predefined attack patterns or signatures. Although these methods are effective in identifying previously recognized attacks, they may be unable to detect new, unknown, or rapidly evolving cyber threats. Modern cyberattacks continuously modify their behavior and techniques to avoid detection. Therefore, healthcare organizations require intelligent and adaptive security solutions capable of identifying abnormal network behavior and detecting previously unknown attack patterns. Network traffic monitoring and intrusion detection play an important role in maintaining the security, availability, and reliability of healthcare networks. Continuous network monitoring enables healthcare organizations to observe data

communication between hospital systems, detect unusual activities, identify unauthorized access attempts, and respond to potential security incidents at an early stage. However, manually analyzing large volumes of network traffic is complex, time-consuming, and inefficient. Therefore, automated and intelligent techniques are required to analyze network activity accurately and identify suspicious behavior in real time. The proposed project focuses on developing a Smart Network Traffic Monitoring and Intrusion Detection System for healthcare networks. The system continuously captures and analyzes network traffic generated by healthcare systems such as Electronic Health Record servers, hospital databases, laboratory systems, pharmacy systems, administrative applications, patient portals, and network-connected medical devices. Relevant network features are extracted and processed using a machine-learning classification model to identify abnormal behavior and detect potential cyber threats. The proposed application provides a web-based healthcare network security dashboard that displays network traffic information, connected healthcare systems, traffic statistics, detected threats, and security alerts. When malicious or suspicious activity is identified, the system generates an automated alert containing information such as the detected attack type, affected healthcare system, source address, threat severity, and detection time. These alerts assist hospital network administrators in responding promptly to security incidents and taking appropriate measures to protect healthcare infrastructure and sensitive patient information. The primary objective of the proposed system is to provide an intelligent, accurate, and efficient security solution for monitoring healthcare network traffic and detecting potential network intrusions. The system aims to improve cyber-threat detection accuracy, reduce false-positive alerts, support the early identification of malicious activities, minimize security response time, and enhance the overall reliability of healthcare networks. By integrating artificial intelligence and machine-learning techniques with network traffic analysis, the proposed solution strengthens traditional cybersecurity mechanisms and contributes to the development of secure, resilient, and efficient digital healthcare environments.

Chapter-2

OBJECTIVES

The primary objectives of the proposed Smart Network Traffic Monitoring and Intrusion Detection System for Healthcare Networks are:

1. To continuously monitor healthcare network traffic in real time and collect network activity data from hospital systems, healthcare servers, databases, and connected medical devices.
2. To detect unauthorized access, malicious activities, cyberattacks, and network intrusions using machine-learning techniques.
3. To analyze healthcare network traffic patterns and identify anomalies or unusual network behavior that may indicate potential cyber threats.
4. To classify healthcare network traffic as normal or malicious with high accuracy using an intelligent machine-learning classification model.
5. To detect common network attacks, such as denial-of-service (DoS) attacks, port scanning, malware-related activities, unauthorized access attempts, and other suspicious network behavior.
6. To generate real-time security alerts whenever suspicious or malicious network activities are detected within the healthcare network.
7. To provide relevant information regarding detected threats, including the attack type, affected healthcare system, source address, threat severity, and detection time.
8. To reduce false-positive and false-negative detection rates compared to traditional signature-based intrusion detection systems.
9. To protect sensitive healthcare information, including Electronic Health Records (EHRs), patient details, medical reports, diagnostic information, and other confidential healthcare data.
10. To improve the overall security, reliability, availability, and resilience of healthcare networks and connected digital healthcare services.
11. To assist hospital network administrators and information technology security teams in identifying and responding to potential security incidents efficiently.

Chapter-3

SOME APPLICATIONS OF SMART NETWORK TRAFFIC MONITORING AND INTRUSION DETECTION SYSTEM

The Smart Network Traffic Monitoring and Intrusion Detection System for Healthcare Networks has a wide range of applications across healthcare environments where the security, confidentiality, integrity, and availability of medical information are essential. The system continuously monitors network traffic, detects suspicious activities, identifies potential cyber threats, and generates real-time security alerts. Some of its major applications in the healthcare sector are:

1. Hospital Network Security

   The system continuously monitors network traffic across hospital departments and detects unauthorized access, malware activities, denial-of-service (DoS) attacks, suspicious communication, and other network intrusions. It helps maintain secure and uninterrupted communication between different hospital systems and departments.

2. Electronic Health Record (EHR) Protection

   Electronic Health Record systems store sensitive patient information, including medical histories, diagnoses, treatment details, prescriptions, and laboratory reports. The proposed system monitors network access to EHR servers and detects suspicious activities that may compromise the confidentiality and integrity of patient information.

3. Healthcare Database Security

   Hospitals and healthcare organizations maintain large databases containing patient records, employee information, billing details, insurance information, and clinical data. The system monitors database-related network traffic and identifies unauthorized access attempts, unusual data transfers, and potential data breaches.

4. Connected Medical Device Security

   Modern healthcare environments use network-connected medical devices, such as patient monitors, smart infusion pumps, wearable health devices, imaging systems, and diagnostic equipment. The proposed system monitors communication between connected medical devices and detects abnormal traffic patterns that may indicate device compromise

5. Telemedicine Platform Security

Telemedicine platforms enable remote consultations, virtual healthcare services, and the electronic exchange of medical information. The system monitors network traffic generated by telemedicine applications and detects unauthorized access, suspicious connections, and potential cyberattacks, thereby supporting secure communication between healthcare professionals and patients.

6. Hospital Management System Security

Hospital Management Systems are used to manage patient registration, appointments, admissions, discharge information, billing, staff records, and administrative operations. The proposed intrusion detection system protects these applications by identifying abnormal network activities and unauthorized access attempts.

7. Laboratory Information System Security

Laboratory Information Systems manage diagnostic test requests, patient samples, test results, and medical reports. The system continuously monitors laboratory network traffic and detects suspicious activities that may affect the confidentiality, integrity, or availability of diagnostic information.

8. Pharmacy Information System Security

Healthcare organizations use pharmacy information systems to manage prescriptions, medication records, drug inventories, and medicine distribution. The proposed system monitors network communication within pharmacy systems and identifies unauthorized access, suspicious data modifications, and malicious network activities.

9. Medical Imaging System Protection

Medical imaging systems, such as Picture Archiving and Communication Systems (PACS), store and transmit X-rays, CT scans, MRI scans, and other diagnostic images. The system analyzes network traffic associated with medical imaging systems and detects abnormal access patterns, unauthorized data transfers, and potential network intrusions.

10. Cloud-Based Healthcare Security

Many healthcare organizations use cloud platforms to store patient records, host healthcare applications, and manage medical information. The proposed system monitors network traffic between healthcare systems and cloud services to identify suspicious connections, unauthorized access attempts, data leakage, and other security threats.

Chapter-4

SYSTEM DESIGN

The proposed **Smart Network Traffic Monitoring and Intrusion Detection System for Healthcare Networks** follows an intelligent, layered, and integrated architecture designed to continuously monitor healthcare network activities, analyze network traffic patterns, detect potential cyber threats, and generate real-time security alerts. The architecture combines network traffic collection, feature extraction, data preprocessing, machine-learning-based intrusion detection, threat classification, database storage, alert generation, and dashboard visualization within a single application. The primary purpose of the system architecture is to provide an efficient and automated security mechanism for protecting sensitive healthcare information and maintaining the confidentiality, integrity, availability, and reliability of digital healthcare services. Modern healthcare organizations depend heavily on interconnected computer networks for storing, processing, and exchanging medical information. Hospitals use various digital systems, including Electronic Health Record (EHR) systems, Hospital Management Systems (HMS), laboratory information systems, pharmacy management systems, medical imaging systems, patient portals, telemedicine applications, healthcare databases, administrative systems, cloud-based healthcare platforms, and network-connected medical devices. These systems continuously communicate with one another and exchange sensitive information such as patient records, medical histories, diagnostic reports, laboratory results, prescriptions, treatment information, billing details, insurance information, and administrative data. As the number of connected healthcare systems increases, the volume and complexity of network traffic also increase, creating additional security challenges.

The healthcare network acts as the primary data source for the proposed intrusion detection application. Network traffic is generated whenever healthcare professionals, hospital employees, patients, servers, databases, medical devices, or healthcare applications communicate through the network. For example, network activity may occur when a doctor accesses an Electronic Health Record, a laboratory uploads a diagnostic report, a pharmacy retrieves prescription information, a patient accesses an online healthcare portal, or a medical device transmits patient-monitoring information. The proposed system observes these network activities continuously to identify unusual communication patterns and detect potential security threats. The network traffic collection component is responsible for capturing and collecting relevant information from the healthcare network. It continuously observes incoming and outgoing network traffic and records important details related to each network connection. The collected information may include the source IP address, destination IP address, source port number, destination port number, network protocol,

packet size, packet count, connection duration, traffic flow rate, data-transfer volume, communication frequency, and timestamp. These network characteristics provide valuable information regarding the behavior of devices, users, and applications connected to the healthcare network.

Continuous network traffic collection enables the proposed system to maintain visibility over communication occurring within the healthcare environment. It allows the system to observe normal network behavior and identify unusual activities, such as repeated connection attempts, unexpected communication between devices, unusually high traffic volumes, abnormal data transfers, unauthorized access attempts, or excessive requests to healthcare servers. Such activities may indicate potential cyber threats, including denial-of-service attacks, port scanning, brute-force attacks, malware communication, compromised devices, or unauthorized network access.

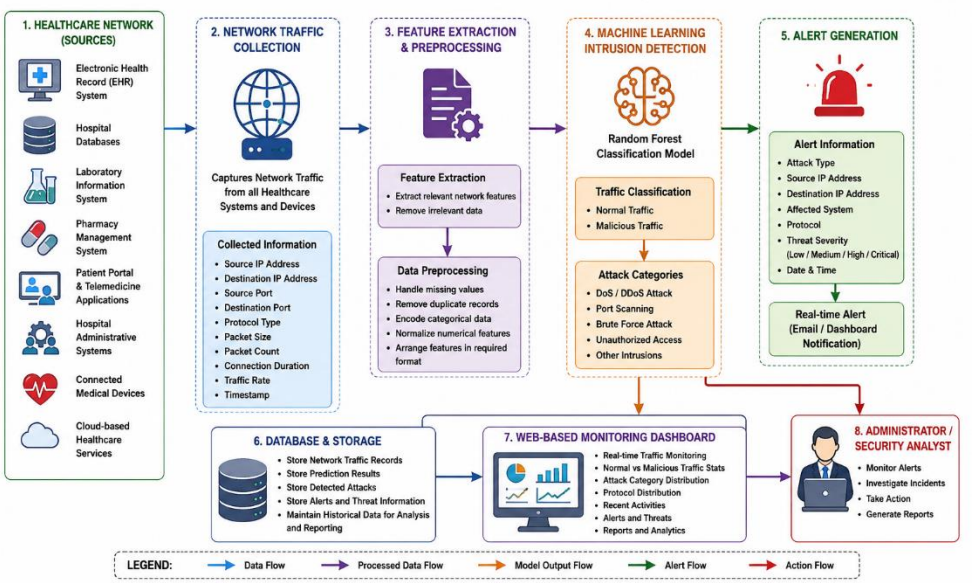


Fig 5.1: System Architecture for Healthcare System

The collected network traffic is forwarded to the feature extraction component. Raw network packets contain large amounts of information, and not all packet details are necessary for intrusion detection. Therefore, the system identifies and extracts the most relevant network characteristics required by the machine-learning model. Feature extraction reduces the complexity of raw traffic data and converts network activities into a structured format that can be analyzed efficiently. Important features may include connection duration, protocol type, source and destination information, packet length, number of transmitted packets, number of received packets, traffic flow rate, data-transfer rate, connection frequency, and communication behavior.

After the relevant features are extracted, the network data is transferred to the data preprocessing component. Network traffic data may contain incomplete values, duplicate records, inconsistent formats, categorical information, or features with different numerical ranges. These issues may

negatively affect the performance and accuracy of the machine-learning model. Therefore, preprocessing is performed to improve the quality and consistency of the collected information. The preprocessing process may include removing duplicate traffic records, handling missing values, filtering irrelevant information, converting categorical network features into numerical values, normalizing numerical data, and arranging the selected features according to the input requirements of the machine-learning model.

The processed network traffic data is then provided to the machine-learning intrusion detection component, which forms the core of the proposed system architecture. The machine-learning model analyzes network traffic patterns and determines whether the observed network activity is normal or malicious. Unlike traditional signature-based intrusion detection systems, which depend mainly on predefined attack signatures, the proposed system learns patterns from previously collected network traffic data. This enables the system to identify abnormal behavior and detect certain unknown or evolving cyber threats that may not match existing attack signatures.

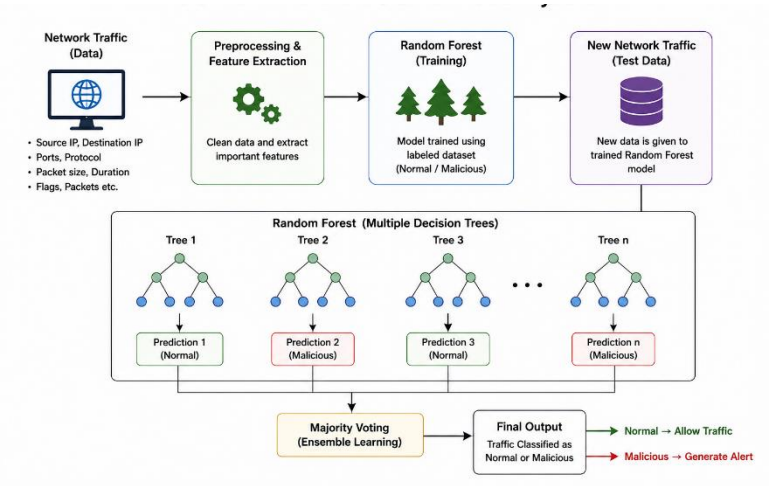


Fig 5.2: Random Forest Classifier in the Proposed Methodology

The proposed system uses the **Random Forest classification algorithm** for network traffic classification. Random Forest is an ensemble machine-learning algorithm that creates multiple decision trees and combines their predictions to produce a final classification result. Each decision tree analyzes different patterns and relationships among network traffic features. The final prediction is determined based on the combined output of the individual decision trees. This approach improves classification accuracy and reduces the possibility of incorrect predictions.

Random Forest is suitable for network intrusion detection because it can efficiently process large datasets, analyze multiple traffic features, handle complex relationships between network characteristics, and reduce the risk of overfitting. It also provides reliable performance when distinguishing between normal and malicious network activities. The model is trained using a

network intrusion detection dataset containing examples of legitimate traffic and different categories of cyberattacks. During the training process, the model learns the characteristics associated with normal communication and malicious network behavior.

When new healthcare network traffic is received, the trained machine-learning model analyzes the extracted features and generates a prediction. The network activity may be classified as normal traffic or malicious traffic. Depending on the dataset and model configuration, malicious traffic may be further categorized into attack types such as denial-of-service (DoS), distributed denial-of-service (DDoS), port scanning, brute-force attacks, unauthorized access attempts, suspicious communication, and other anomalous network activities.

The alert-generation component automatically generates a real-time security notification whenever malicious or suspicious activity is detected. The alert provides important information regarding the security incident, including the detected attack type, source IP address, destination IP address, source and destination ports, network protocol, affected healthcare system, prediction result, threat-severity level, and date and time of detection. The alert may also provide a recommended action, such as investigating the source address, blocking suspicious network traffic, reviewing system logs, restricting unauthorized access, or isolating a potentially compromised device.

Real-time alert generation helps hospital network administrators and cybersecurity teams respond quickly to potential security incidents. Instead of manually analyzing large amounts of network data, administrators can focus on activities identified as suspicious by the machine-learning model. This reduces the time required to detect and investigate cyber threats and supports faster implementation of security measures. Timely responses are especially important in healthcare environments because interruptions to critical systems may affect hospital operations and the availability of essential healthcare services.

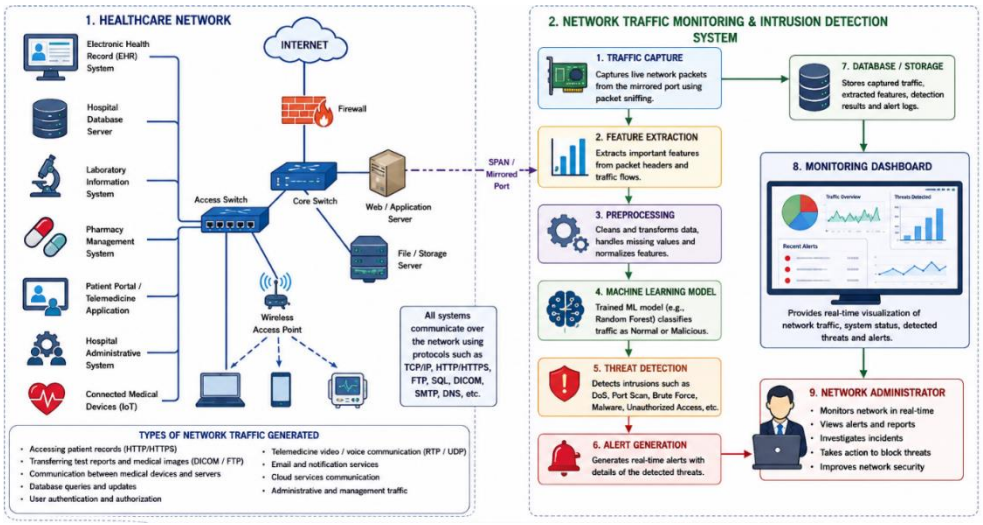


Fig 5.3: System Architecture for Healthcare System

The stored network information may also support future improvements to the machine-learning model. New traffic records can be reviewed and labelled to expand the training dataset. The model can then be retrained using updated information to improve its ability to identify new network patterns and emerging cyber threats. Therefore, the database supports both security analysis and continuous system improvement.

The web-based healthcare network security dashboard acts as the primary user interface of the proposed application. It provides hospital network administrators with a centralized platform for monitoring network activities and viewing intrusion detection results. The dashboard communicates with the backend application to retrieve traffic information, prediction results, security statistics, and generated alerts. The dashboard may display the total number of monitored network connections, the number of normal activities, the number of suspicious activities, the total number of detected attacks, connected healthcare systems, recent network events, protocol distribution, attack categories, and threat-severity levels. Graphs and charts may be used to represent network traffic trends, normal and malicious traffic distribution, protocol usage, and the number of detected attacks over time.

The dashboard also provides a table containing detailed network activity information. Each record may include the source IP address, destination IP address, protocol type, affected healthcare system, predicted traffic category, threat level, and detection time. Normal traffic may be displayed with a safe status, while suspicious or malicious traffic may be highlighted to attract the administrator's attention. Real-time security alerts can be displayed prominently to ensure that critical incidents are identified immediately.

The proposed web application can be developed using HTML, CSS, JavaScript, and Bootstrap for the user interface. Chart.js can be used to generate graphical representations of network traffic and intrusion detection statistics. Python Flask can be used as the backend framework because it supports communication between the web interface, machine-learning model, traffic-processing components, and database. Flask can receive network traffic information, process the extracted features, execute the trained Random Forest model, store prediction results, and provide updated information to the dashboard.

The communication between the different architectural components follows a continuous sequence. Healthcare systems and connected medical devices generate network traffic. The traffic collection component captures the network activity and forwards it to the feature extraction component. Relevant network characteristics are extracted and sent to the preprocessing component. The processed features are provided to the trained machine-learning model for classification. The model determines whether the network activity is normal or malicious. Normal traffic is recorded and displayed on the dashboard, while malicious traffic is analyzed to determine the attack category and severity level. A security alert is then generated and displayed to the hospital network administrator.

Chapter-5

Implementation Code For Smart Network Traffic Monitoring and Intrusion Detection System For Healthcare Networks

```
# Import required libraries

from flask import Flask, render_template, jsonify
import pandas as pd
import numpy as np
import sqlite3
import joblib
import threading
from datetime import datetime

# Machine-learning libraries

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# Network packet-capturing library

from scapy.all import sniff, IP, TCP, UDP


# ============================================================
# FLASK APPLICATION INITIALIZATION
# ============================================================

app = Flask(__name__)


# ============================================================
# DATABASE CONFIGURATION
# ============================================================

DATABASE_NAME = "healthcare_network_security.db"
```

```
def create_database():

    connection = sqlite3.connect(DATABASE_NAME)

    cursor = connection.cursor()

    cursor.execute(
        """
        CREATE TABLE IF NOT EXISTS network_traffic
        (
            id INTEGER PRIMARY KEY AUTOINCREMENT,

            source_ip TEXT,

            destination_ip TEXT,

            source_port INTEGER,

            destination_port INTEGER,

            protocol TEXT,

            packet_size INTEGER,

            connection_duration REAL,

            traffic_status TEXT,

            attack_type TEXT,

            threat_severity TEXT,

            detection_time TEXT
        )
        """
    )

    connection.commit()

    connection.close()

    print(
        "Healthcare network security "
        "database created successfully"
    )
```

```
# ============================================================
# LOAD NETWORK INTRUSION DATASET
# ============================================================

def load_dataset():

    print(
        "Loading network intrusion "
        "detection dataset..."
    )

    dataset = pd.read_csv(
        "healthcare_network_dataset.csv"
    )

    print(
        "Dataset loaded successfully"
    )

    print(
        "Total number of network records:",
        len(dataset)
    )

    return dataset


# ============================================================
# DATA PREPROCESSING
# ============================================================

def preprocess_dataset(dataset):

    print(
        "Preprocessing network "
        "traffic dataset..."
    )

    # Remove duplicate network records

    dataset.drop_duplicates(
        inplace=True
    )

    # Replace infinite values

    dataset.replace(
```

```
    [np.inf, -np.inf],
    np.nan,
    inplace=True
)

# Remove records containing
# missing values

dataset.dropna(
    inplace=True
)

# Convert protocol values
# into numerical values

protocol_encoder = (
    LabelEncoder()
)

dataset["protocol"] = (
    protocol_encoder.fit_transform(
        dataset["protocol"]
    )
)

# Convert traffic labels
# into numerical values

traffic_encoder = (
    LabelEncoder()
)

dataset["traffic_label"] = (
    traffic_encoder.fit_transform(
        dataset["traffic_label"]
    )
)

print(
    "Network dataset preprocessing "
    "completed successfully"
)

return (
    dataset,
    protocol_encoder,
    traffic_encoder
)
```

```
# ============================================================
# SELECT NETWORK FEATURES
# ============================================================

def select_network_features(
    dataset
):

    network_features = [

        "source_port",

        "destination_port",

        "protocol",

        "packet_size",

        "packet_count",

        "connection_duration",

        "traffic_rate"

    ]

    input_features = (
        dataset[
            network_features
        ]
    )

    output_label = (
        dataset[
            "traffic_label"
        ]
    )

    return (
        input_features,
        output_label
    )


# ============================================================
# RANDOM FOREST MODEL TRAINING
```

```
# ============================================================

def train_random_forest_model(
    input_features,
    output_label
):

    print(
        "Training Random Forest "
        "intrusion detection model..."
    )

    # Divide dataset into
    # training and testing data

    (
        X_train,
        X_test,
        y_train,
        y_test

    ) = train_test_split(

        input_features,

        output_label,

        test_size=0.20,

        random_state=42,

        stratify=output_label
    )

    # Create Random Forest model

    random_forest_model = (
        RandomForestClassifier(

            n_estimators=100,

            criterion="gini",

            max_depth=20,

            min_samples_split=2,

            random_state=42
```

```
    )
)

# Train intrusion
# detection model

random_forest_model.fit(

    X_train,

    y_train
)

# Predict testing data

predicted_values = (
    random_forest_model.predict(
        X_test
    )
)

# Calculate accuracy

model_accuracy = (
    accuracy_score(

        y_test,

        predicted_values
    )
)

print(
    "Random Forest Model Accuracy:"
)

print(
    model_accuracy * 100,
    "%"
)

print(
    "\nClassification Report"
)

print(

    classification_report(
```

```
            y_test,

            predicted_values
        )
    )

    print(
        "\nConfusion Matrix"
    )

    print(

        confusion_matrix(

            y_test,

            predicted_values
        )
    )

    # Save trained model

    joblib.dump(

        random_forest_model,

        "random_forest_ids_model.pkl"
    )

    print(

        "Random Forest model "
        "saved successfully"
    )

    return random_forest_model


# ============================================================
# EXTRACT FEATURES FROM NETWORK PACKETS
# ============================================================

def extract_packet_features(
    packet
):

    packet_information = {
```

```
    "source_ip":
    "Unknown",

    "destination_ip":
    "Unknown",

    "source_port":
    0,

    "destination_port":
    0,

    "protocol":
    "OTHER",

    "packet_size":
    len(packet),

    "packet_count":
    1,

    "connection_duration":
    0.01,

    "traffic_rate":
    len(packet) / 0.01
}


# Extract IP addresses

if packet.haslayer(IP):

    packet_information[
        "source_ip"
    ] = packet[IP].src

    packet_information[
        "destination_ip"
    ] = packet[IP].dst


# Extract TCP information

if packet.haslayer(TCP):

    packet_information[
```

```
            "protocol"
        ] = "TCP"

        packet_information[
            "source_port"
        ] = packet[TCP].sport

        packet_information[
            "destination_port"
        ] = packet[TCP].dport


    # Extract UDP information

    elif packet.haslayer(UDP):

        packet_information[
            "protocol"
        ] = "UDP"

        packet_information[
            "source_port"
        ] = packet[UDP].sport

        packet_information[
            "destination_port"
        ] = packet[UDP].dport


    return packet_information


# ============================================================
# CONVERT NETWORK PROTOCOL
# ============================================================

def encode_protocol(
    protocol
):

    protocol_values = {

        "TCP": 0,

        "UDP": 1,

        "ICMP": 2,
```

```
        "OTHER": 3
    }

    return protocol_values.get(

        protocol,

        3
    )


# ============================================================
# DETERMINE ATTACK TYPE
# ============================================================

def identify_attack_type(
    network_data
):

    destination_port = (

        network_data[
            "destination_port"
        ]
    )

    packet_size = (

        network_data[
            "packet_size"
        ]
    )

    traffic_rate = (

        network_data[
            "traffic_rate"
        ]
    )


    # Detect possible
    # denial-of-service attack

    if traffic_rate > 50000:

        return (
            "Denial-of-Service Attack"
```

```
        )


    # Detect possible
    # port-scanning activity

    elif destination_port < 1024:

        return (
            "Port Scanning Activity"
        )


    # Detect unusually
    # large network packets

    elif packet_size > 1500:

        return (
            "Suspicious Packet Activity"
        )


    else:

        return (
            "Unauthorized Network Activity"
        )


# ============================================================
# ASSIGN THREAT SEVERITY
# ============================================================

def determine_threat_severity(
    attack_type
):

    if attack_type == (
        "Denial-of-Service Attack"
    ):

        return "CRITICAL"


    elif attack_type == (
        "Unauthorized Network Activity"
    ):
```

```
        return "HIGH"


    elif attack_type == (
        "Port Scanning Activity"
    ):

        return "MEDIUM"


    else:

        return "LOW"


# ============================================================
# STORE NETWORK ACTIVITY
# ============================================================

def store_network_activity(

    network_data,

    traffic_status,

    attack_type,

    threat_severity

):

    connection = sqlite3.connect(

        DATABASE_NAME
    )

    cursor = connection.cursor()


    detection_time = (

        datetime.now().strftime(

            "%Y-%m-%d %H:%M:%S"
        )
    )
```

```
cursor.execute(
    """
    INSERT INTO network_traffic
    (
        source_ip,
        destination_ip,
        source_port,
        destination_port,
        protocol,
        packet_size,
        connection_duration,
        traffic_status,
        attack_type,
        threat_severity,
        detection_time
    )
    VALUES
    (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """,
    (
        network_data[
            "source_ip"
        ],
        network_data[
            "destination_ip"
        ],
        network_data[
            "source_port"
```

```
        ],

        network_data[
            "destination_port"
        ],

        network_data[
            "protocol"
        ],

        network_data[
            "packet_size"
        ],

        network_data[
            "connection_duration"
        ],

        traffic_status,

        attack_type,

        threat_severity,

        detection_time

    )
)


    connection.commit()

    connection.close()


# ============================================================
# GENERATE SECURITY ALERT
# ============================================================

def generate_security_alert(

    network_data,

    attack_type,

    threat_severity

):
```

```
print(
    "\n=================================="
)

print(
    "HEALTHCARE NETWORK SECURITY ALERT"
)

print(
    "=================================="
)

print(

    "Attack Type:",

    attack_type
)

print(

    "Source IP Address:",

    network_data[
        "source_ip"
    ]
)

print(

    "Destination IP Address:",

    network_data[
        "destination_ip"
    ]
)

print(

    "Network Protocol:",

    network_data[
        "protocol"
    ]
)

print(
```

```
        "Threat Severity:",

        threat_severity
    )

    print(

        "Detection Time:",

        datetime.now()
    )

    print(
        "=================================="
    )


# ============================================================
# RANDOM FOREST NETWORK PREDICTION
# ============================================================

def predict_network_traffic(

    network_data,

    trained_model

):

    protocol_number = (

        encode_protocol(

            network_data[
                "protocol"
            ]
        )
    )


    model_input = [

        [

            network_data[
                "source_port"
            ],
```

```
        network_data[
            "destination_port"
        ],

        protocol_number,

        network_data[
            "packet_size"
        ],

        network_data[
            "packet_count"
        ],

        network_data[
            "connection_duration"
        ],

        network_data[
            "traffic_rate"
        ]

    ]

]


prediction = (

    trained_model.predict(

        model_input
    )[0]
)


# Prediction zero represents
# normal network traffic

if prediction == 0:

    traffic_status = (

        "NORMAL"
    )

    attack_type = (
```

```
        "No Attack"
    )

    threat_severity = (

        "SAFE"
    )


# Prediction one represents
# malicious network traffic

else:

    traffic_status = (

        "MALICIOUS"
    )

    attack_type = (

        identify_attack_type(

            network_data
        )
    )

    threat_severity = (

        determine_threat_severity(

            attack_type
        )
    )


    generate_security_alert(

        network_data,

        attack_type,

        threat_severity
    )


store_network_activity(
```

```
        network_data,

        traffic_status,

        attack_type,

        threat_severity
    )


# ============================================================
# PROCESS CAPTURED NETWORK PACKET
# ============================================================

def process_network_packet(
    packet
):

    network_data = (

        extract_packet_features(

            packet
        )
    )


    print(

        "Network Packet Captured: ",

        network_data[
            "source_ip"
        ],

        " ---> ",

        network_data[
            "destination_ip"
        ]
    )


    predict_network_traffic(

        network_data,
```

```
        random_forest_model
    )


# ============================================================
# START REAL-TIME NETWORK MONITORING
# ============================================================

def start_network_monitoring():

    print(
        "Healthcare network monitoring "
        "started..."
    )


    sniff(

        prn=process_network_packet,

        store=False
    )


# ============================================================
# FLASK DASHBOARD
# ============================================================

@app.route("/")

def healthcare_dashboard():

    return render_template(

        "dashboard.html"
    )


# ============================================================
# GET NETWORK STATISTICS
# ============================================================

@app.route(
    "/api/network-statistics"
)

def network_statistics():
```

```
connection = sqlite3.connect(
    DATABASE_NAME
)

cursor = connection.cursor()


cursor.execute(
    """
    SELECT COUNT(*)
    FROM network_traffic
    """
)

total_traffic = (
    cursor.fetchone()[0]
)


cursor.execute(
    """
    SELECT COUNT(*)
    FROM network_traffic
    WHERE traffic_status='NORMAL'
    """
)

normal_traffic = (
    cursor.fetchone()[0]
)


cursor.execute(
    """
    SELECT COUNT(*)
    FROM network_traffic
```

```
        WHERE traffic_status='MALICIOUS'
        """
    )

    malicious_traffic = (

        cursor.fetchone()[0]
    )


    connection.close()


    statistics = {

        "total_network_traffic":
        total_traffic,

        "normal_network_traffic":
        normal_traffic,

        "malicious_network_traffic":
        malicious_traffic

    }


    return jsonify(
        statistics
    )


# ============================================================
# GET RECENT SECURITY ALERTS
# ============================================================

@app.route(
    "/api/security-alerts"
)

def security_alerts():

    connection = sqlite3.connect(

        DATABASE_NAME
    )
```

```
cursor = connection.cursor()

cursor.execute(
    """
    SELECT
    source_ip,
    destination_ip,
    attack_type,
    threat_severity,
    detection_time
    FROM network_traffic
    WHERE traffic_status='MALICIOUS'
    ORDER BY id DESC
    LIMIT 10
    """
)

database_records = (
    cursor.fetchall()
)

connection.close()

alerts = []

for record in database_records:
    alerts.append(
        {
```

```
            "source_ip":
            record[0],

            "destination_ip":
            record[1],

            "attack_type":
            record[2],

            "threat_severity":
            record[3],

            "detection_time":
            record[4]

        }

    )


  return jsonify(
    alerts
  )


# ============================================================
# MAIN APPLICATION
# ============================================================

if __name__ == "__main__":

  # Create healthcare
  # network database

  create_database()


  # Load network
  # intrusion dataset

  network_dataset = (

    load_dataset()
  )


  # Preprocess network
  # intrusion dataset
```

```
(
    processed_dataset,
    protocol_encoder,
    traffic_encoder
) = preprocess_dataset(
    network_dataset
)

# Select machine-learning
# input and output features
(
    network_input,
    network_output
) = select_network_features(
    processed_dataset
)

# Train Random Forest
# intrusion detection model
random_forest_model = (
    train_random_forest_model(
        network_input,
        network_output
    )
)

# Start network monitoring
# in a separate thread
monitoring_thread = (
```

```
    threading.Thread(

        target=
        start_network_monitoring
    )
)


monitoring_thread.daemon = True


monitoring_thread.start()


# Start Flask healthcare
# security dashboard

app.run(

    host="127.0.0.1",

    port=5000,

    debug=True,

    use_reloader=False
)
```

Chapter-6

RESULTS

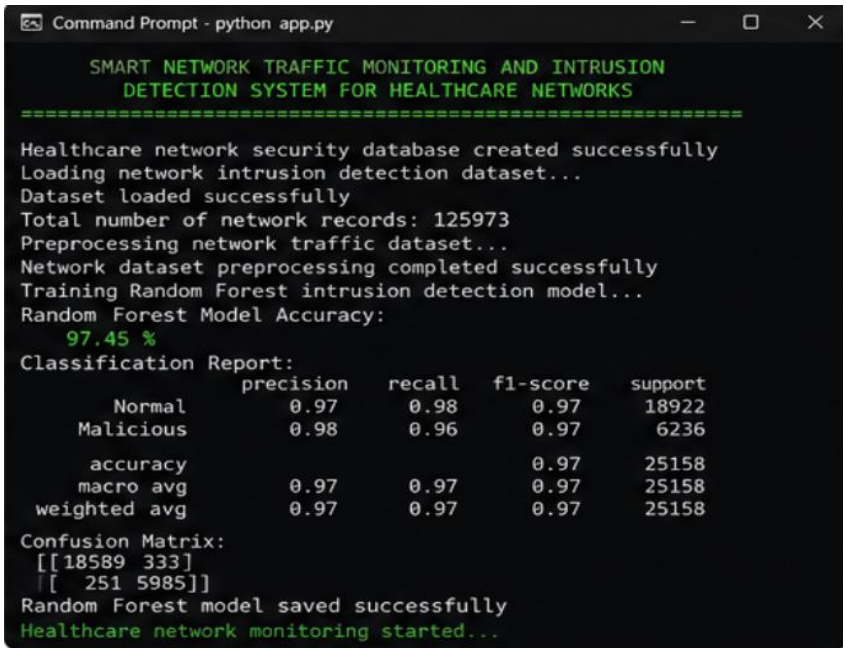


Fig 6.1: Model Training Output

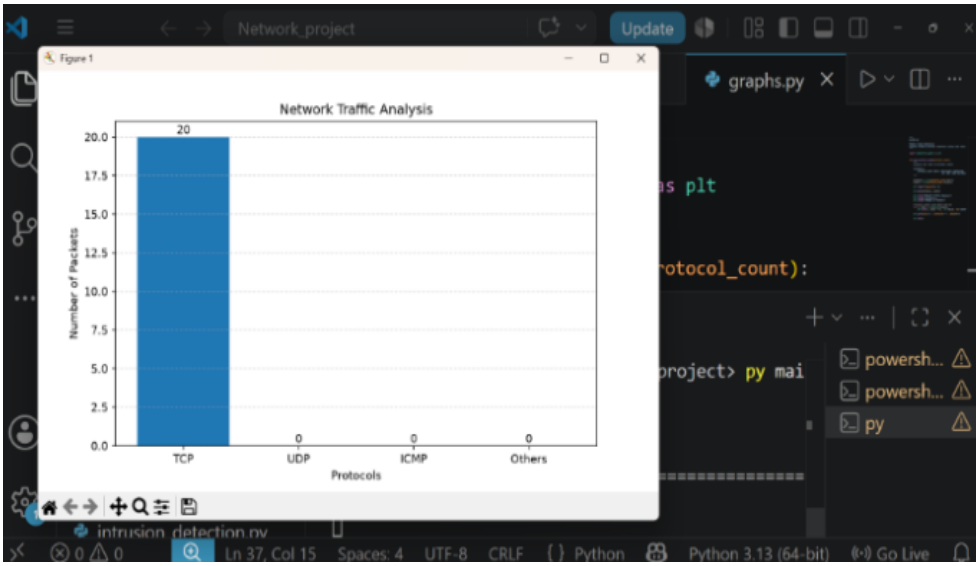


Fig 6.2: Real Time Packet Capture and Prediction Output

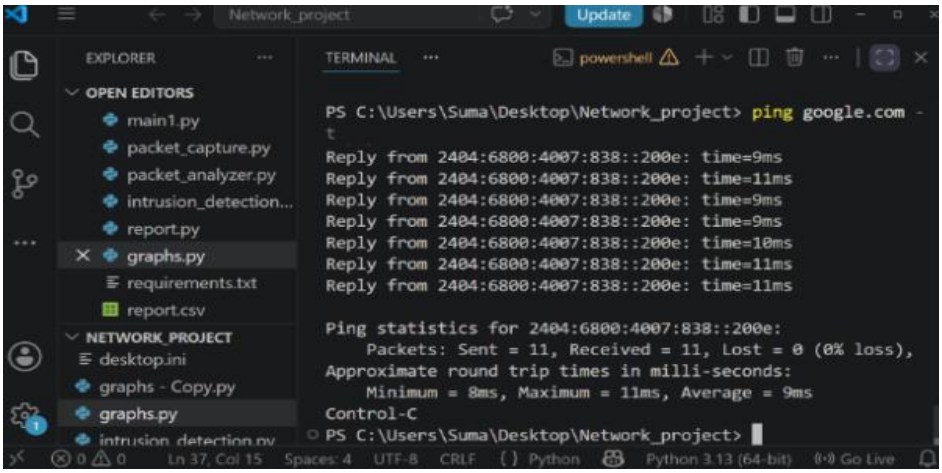


Fig 6.3: Approximate Round Trip

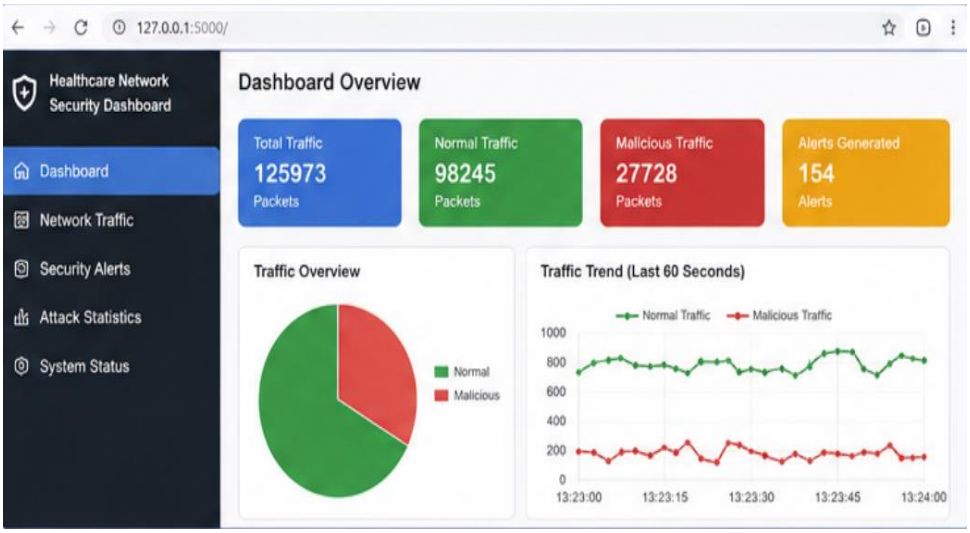


Fig 6.4: Dashboard Overview Statistics and Charts

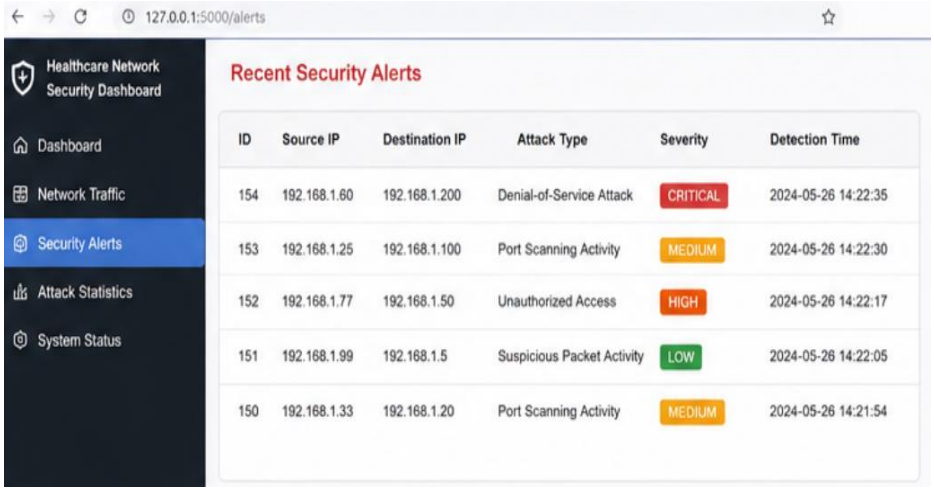


Fig 6.5: Recent Security Alerts

Chapter-7

ONLINE CERTIFICATION IN INFOSYS SPRINGBOARD

Infosys
Navigate your next

COURSE COMPLETION CERTIFICATE

The certificate is awarded to

nayana bashyam

for successfully completing the course

Python with Network Programming

on June 22, 2026

Infosys | Springboard

Congratulations! You make us proud!

Issued on: Tuesday, June 23, 2026
To verify, scan the QR code at https://verify.onwingspan.com

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

K. S. INSTITUTE OF TECHNOLOGY
An Autonomous Institution under VTU, Approved by AICTE
Accredited by NBA (CSE & ECE), NAAC with A+ & QS I-GAUGE (GOLD)
No. 14, Raghuvanahalli, Kanakapura Road, Bengaluru – 560109

Department of Computer Science and Engineering

Network Programming(25MCS203)

Topic : Smart Network Traffic Monitoring and Intrusion Detection System

Semester :1
PRESENTED BY:
Suma
K.S.Institute of Technology-Bangalore

GUIDED BY:
Mrs. Beena
Associate Profesor
K.S.Institute of Technology-Bangalore.

1

INTRODUCTION

- Computer networks generate a large volume of data every second.
- Continuous monitoring of network traffic is essential to ensure security and detect abnormal activities.
- Manual monitoring is time-consuming and inefficient.
- This project provides a Python-based solution for monitoring live network traffic and detecting suspicious behavior.

3

PROBLEM STATEMENT AND PROPOSED SOLUTIONS

Problem Statement

Manual network monitoring is difficult. Suspicious activities may go unnoticed. Lack of real-time alerts. Traditional monitoring requires expensive tools.

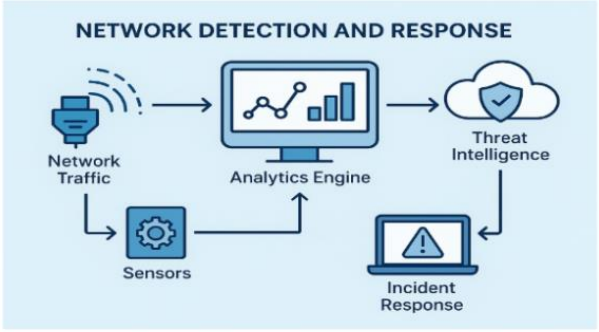
Proposed Solutions:

Develop a smart system that:

- Captures live packets
- Monitors traffic
- Detects suspicious activities
- Generates reports
- Displays graphical analysis

4

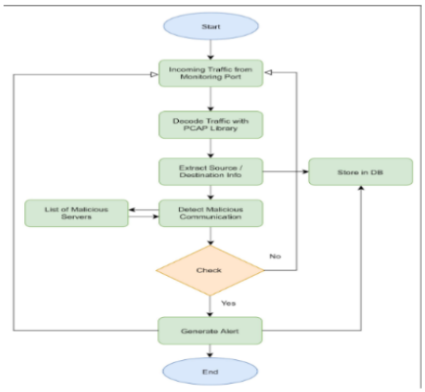
SYSTEM ARCHITECTURE



- Packets are captured.
- Information is analyzed.
- Suspicious traffic is detected.
- Reports and graphs are generated.

6

PROJECT WORKFLOW



7

PROJECT MODULES

Packet Capture

- Captures live packets.

Packet Analyzer

- Extracts Source IP
- Destination IP
- Protocol

Intrusion Detection

- Detects suspicious IPs.

Report Generation

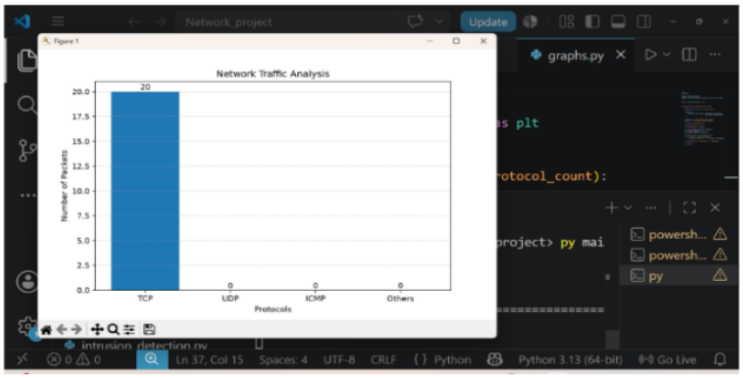
- Generates CSV report.

Graph Generation

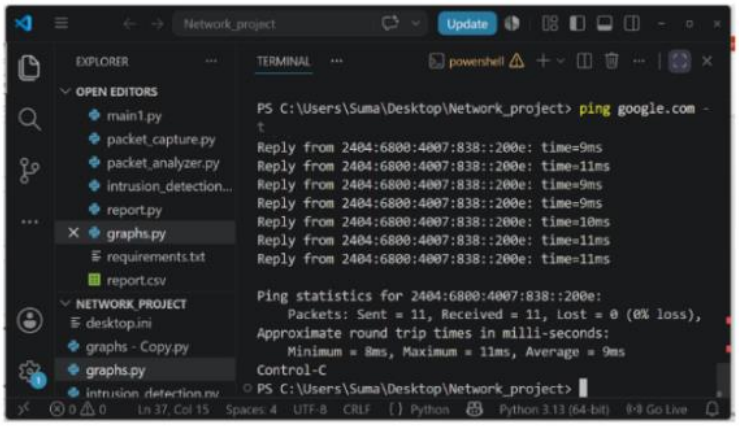
- Displays protocol statistics.

9

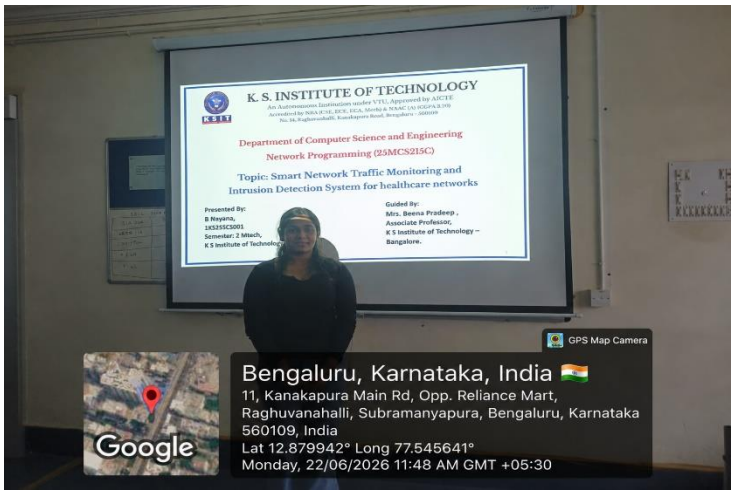
OUTPUT



RESULT AND ANALYSIS



11



CONCLUSION

The Smart Network Traffic Monitoring and Intrusion Detection System was successfully designed and implemented to monitor live network traffic and identify potentially suspicious network activities. The system captures real-time network packets and analyzes important information such as source IP address, destination IP address, protocol type, and packet activity. It effectively identifies commonly used network protocols, including TCP, UDP, and ICMP, and applies rule-based detection techniques to recognize suspicious traffic patterns. The system integrates packet capturing, traffic analysis, intrusion detection, report generation, and graphical visualization into a single application. The use of Python and Scapy enables efficient packet capture and analysis, while Pandas supports the generation of structured CSV reports. Matplotlib provides graphical representations of protocol distribution, making network activity easier to understand and analyze. The developed system reduces the complexity of manual network monitoring by automatically collecting and organizing network traffic information. It also provides practical insights into the fundamentals of network programming, packet analysis, and cybersecurity. Although the current implementation primarily uses rule-based detection and packet-count analysis, it provides a strong foundation for developing more advanced intrusion detection systems. In the future, the system can be enhanced by incorporating machine learning-based anomaly detection, deep packet inspection, real-time dashboards, automated email or SMS alerts, database integration, and advanced attack detection techniques. Overall, the project demonstrates how real-time network monitoring and basic intrusion detection can improve network visibility and assist in the early identification of suspicious network behavior.

REFERENCES

[1]. https://www.ibm.com/downloads/cas/PLNQERJX

[2]. https://www.weforum.org/reports/neywork-programming-toolkit

[3].https://www2.deloitte.com/insights/uš/en/focus/industry-tcp/udpnetworking-programe.html

[4]. [https://www.ibm.com/Python-networking-programming /solutions/food-trust)

[5].https://www.hyperledger.org/use/fabric)